

UNIVERSIDADE FEDERAL DE VIÇOSA
CAMPUS FLORESTAL

MARIA PAULA LOPES SILVA

Relatório Final PIBIC:
Inteligência Artificial e Lógica
Matemática

Relatório Final de Iniciação Científica
apresentado ao Programa Institucional
de Bolsas de Iniciação Científica (PI-
BIC).

Orientador: Prof. Sérgio Henrique
Nogueira

Florestal - MG

2024

Sumário

1	Capítulo 1: Introdução – Uma Pequena Viagem pelo Reino da Lógica e da Computação	4
2	Capítulo 2: Dedução, Indução e Paradóxos	4
3	Capítulo 3: Máquinas que Aprendem	5
3.1	Jogo NIM	5
3.1.1	Regras gerais	5
3.1.2	Configurações	5
3.2	A Teoria do XOR	6
3.3	Como Calcular o XOR	6
3.4	Variáveis que Influenciam o Jogo:	9
3.4.1	1- Número de Fileiras:	9
3.4.2	2- Ordem dos Jogadores:	9
3.5	Outra Estratégia	9
3.6	Jogo de Tabuleiro	11
3.6.1	Como Funciona?	11
3.6.2	Objetivo	11
3.6.3	Regras Gerais	11
3.6.4	Como o Computador Aprende?	12
4	Capítulo 4: Máquinas de calcular do arco da velha	14
4.1	Somando com água	14
4.2	Números Binários	14
4.2.1	Conversão de número decimal para binário	14
4.2.2	Conversão de número binário para sistema decimal	15
4.2.3	Operações com números binários	16
4.2.4	Circuitos Somadores → Half Adder	17
4.2.5	Tabela Verdade em um Meio Somador	17
4.3	Calculadora do Fred Flintstone	18
4.4	Lata de Calcular	19
4.5	A Calculadora Parabólica	19
4.6	Pelas Barbas de Naipier	19
4.7	A Origem dos Computadores	20
4.8	Funcionamento dos Computadores Analógicos	21

5	Capítulo 5: Conversando com Máquinas	22
5.1	A Teoria da Caixa Preta	22
5.2	Objetivo	22
5.3	O Conceito de Autômato	22
5.4	Lógica da Máquina	22
5.4.1	Como Cada Letra Afeta o Estado da Lâmpada	22
5.4.2	Outras Máquinas e Outras Linguagens	23
6	Capítulo 6: Autômatos	25
6.1	Sistema de Estados Finitos e Determinísticos	26
6.2	Lema do Bombardeio	27
7	Capítulo 7: Como uma Máquina Pode se Lembrar	28
7.1	Máquinas que reconhecem linguagens	28
7.2	O Autômato de Pilha	28
7.3	Memória Eletrônica	30
7.3.1	Modelo Relê	31
7.3.2	Flip-Flops	33
7.3.3	Flip-Flop Set and Reset	33
8	Capítulo 8: Gramática Combina com Matemática?	38
8.1	Procedimentos VS Algoritmos	38
9	Capítulo 9: O Mais Poderoso de Todos os Computadores	38
9.1	Máquina de Turing	38
9.2	RAM- Máquina com Acesso Aleatório	39
10	Capítulo 10: A Vida Sexual das Máquinas	39
10.1	Autômatos Celulares	39
10.2	Jogo da Vida	39
11	Capítulo 11: O que a Máquina Faz, Mas Não Deveria Fazer	42
11.1	Classes P e NP	44
11.2	Máquina de Turing	44
11.2.1	Determinística	44
11.2.2	Não-Determinísticos	44
12	Capítulo 12: O Que As Máquinas Nunca Farão	45
12.1	Máquina DIN-DIN	46
12.1.1	Sentenças	46
12.2	Castor Ocupado	47

12.3 O Problema da Parada	49
12.4 O Jogo do MU	50
12.4.1 Regras	50
12.5 Copinhos de Café	51
12.5.1 Objetivo	51
13 Considerações Finais	52
14 Agradecimento	52
15 Referências Das Pesquisas	52
15.1 Jogo NIM	52
15.2 Meio Somador	52
15.3 Linguagens	52
15.4 Autômatos Finitos Determinísticos	52
15.5 Relês	53
15.6 As Máquinas de Turing	53
15.7 Autômatos Celulares	53

1 Capítulo 1: Introdução – Uma Pequena Viagem pelo Reino da Lógica e da Computação

No capítulo 1, foi estudado sobre o quanto os computadores estão presentes no nosso dia a dia, principalmente no ensino.

Para compreender o funcionamento das máquinas, pode-se recorrer a exemplos simples, como jogos de lógica, de tentativa e erro, de recompensas, entre outros.

2 Capítulo 2: Dedução, Indução e Paradóxsos

Nesta parte dos estudos, foram aprendidos os conceitos de **Dedução** e **Indução** e como eles se relacionam com a lógica.

Os dois conceitos têm formas diferentes de chegar a uma conclusão/resultado.

A **Dedução**, é o tipo de raciocínio mais direto e assertivo. Neste caso, se as premissas (suposições) são **verdadeiras**, a conclusão é inevitável.

Exemplo:

"Todos os cães têm patas. O cachorro da vizinha é um cão. Logo, o cachorro da vizinha tem patas." A lógica aqui é mais sólida e confiável.

Já a **Indução** parte do princípio da **observação** e **generalização**.

Exemplo:

"Todas as maçãs que vi até agora são vermelhas, então todas devem ser vermelhas." É óbvio que a lógica não é bem assim, pois nem todas as maçãs são vermelhas.

Nesse tipo de pensamento, as conclusões nunca são cem por cento concretas, mas podem ser usadas pra tirar conclusões mais rápidas.

Os **Paradóxsos** servem pra tirar conclusões, só que os pensamentos são contraditórios.

Exemplo:

"Eu estou mentindo." Se a pessoa está mentindo, então ela não está mentindo, o que cria uma contradição sem fim.

Além do *Paradoxo do Mentiroso*, há muitos outros, como o *Paradoxo da Exceção*, que diz: *"Toda regra tem uma exceção"* que já se contradiz, pois, ao dizer isso, a própria regra pode ser quebrada.

E por esses motivos, os **paradoxos** devem ser **ao máximo evitados**, as informações e passos devem estar claros e evidentes às máquinas, para que não se confundam.

3 Capítulo 3: Máquinas que Aprendem

Essa semana, foi lido sobre o aprendizado das máquinas, que basicamente é como as máquinas podem melhorar suas habilidades com o tempo, como uma evolução natural.

Exemplo: Um computador que começa jogando de forma aleatória, mas com o tempo, aprende de acordo com suas experiências de vitória e derrota.

Quando a máquina faz uma jogada que tem bons resultados (vitórias), ela registra essa informação como *útil* para reutilizá-las nas próximas jogadas. Já quando ela tem resultados/jogadas ruins (que as levam a derrota), a máquina evita repeti-las.

Esse tipo de aprendizado é como se fosse um **banco de dados** que ajuda o computador a melhorar e se tornar mais eficiente ao longo do tempo.

Com o tempo, o computador acumula um grande número de informações e consegue avançar de níveis, podendo superar jogadores humanos inclusive.

Todo esse procedimento é baseado em TENTATIVA E ERRO, que treinam o PC ao longo das partidas.

3.1 Jogo NIM

O *NIM* é um jogo de estratégia pra dois jogadores. Ele é jogado com várias fileiras de objetos (como palitos, pedras ou moedas). Os jogadores se revezam removendo qualquer número de objetos de uma única fileira por turno. Quem pegar o último objeto vence ou perde, dependendo da regra de vitória escolhida.

3.1.1 Regras gerais

Vitória:

Regra Normal: Vence quem pegar o último objeto.

Regra Misère: Perde quem pegar o último objeto.

***OBS:** As regras do jogo determinam as estratégias adotadas e consequentemente o resultado final.

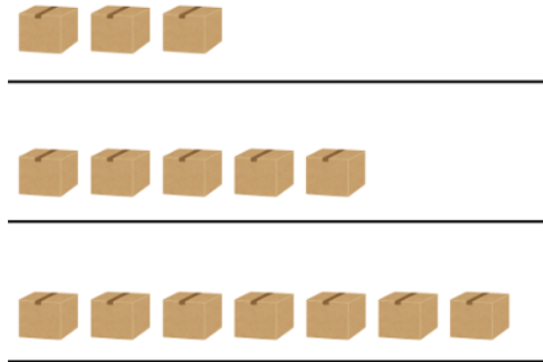
3.1.2 Configurações

Decida o número de fileiras e a quantidade de objetos em cada uma.

Exemplo:

3 fileiras com 3, 5 e 7 objetos.

Figura 1: Exemplo de configuração NIM.



Revezamento: Em cada turno, um jogador remove qualquer quantidade de objetos de uma única fileira.

3.2 A Teoria do XOR

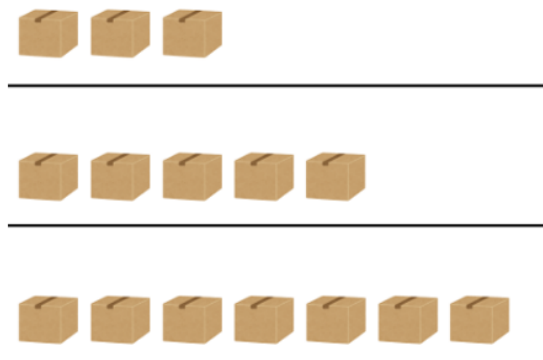
O XOR é uma operação lógica usada para calcular a posição do jogo. Ele indica se a posição atual é vantajosa ou não.

3.3 Como Calcular o XOR

Converta o número de objetos de cada fileira para binário e faça a operação XOR linha por linha.

Exemplo: Fileiras com 3, 5 e 7 objetos

Figura 2: Exemplo do XOR.



Fileira n° 1 (3): 011

Fileira n° 2 (5): 101

Fileira n° 3 (7): 111

XOR:

1- XOR entre 3 e 5:

Figura 3: XOR 3 e 5.

Bit de 3	Bit de 5	XOR
0	1	1
1	0	1
1	1	0

Resultado: 110

2- XOR entre 3, 5 e 7:

Figura 4: XOR 3 e 5.

Bit de 3	Bit de 5	XOR
0	1	1
1	0	1
1	1	0

Figura 5: XOR 6 e 7.

Bit de 6	Bit de 7	XOR
1	1	0
1	1	0
0	1	1

Resultado: 001

Posições Seguras: O resultado do XOR é 0.

Estar em uma posição **segura** significa que o jogador que acabou de jogar pode **forçar** o adversário a uma posição **insegura** (se jogar corretamente).

Quando você entrega uma posição segura ao oponente, você o **força** a cometer erros (se ele não souber jogar perfeitamente).

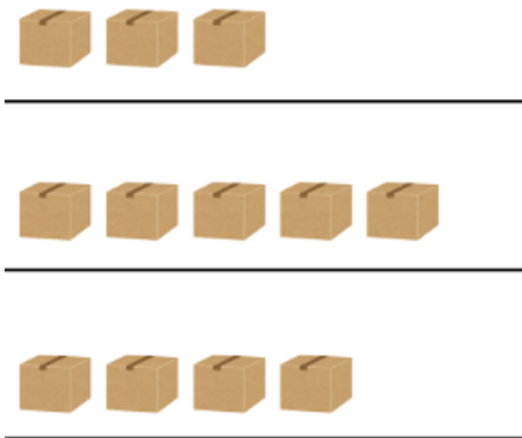
Posição Inseguras: O resultado do XOR não é 0.

Estar em uma posição **insegura** significa que o adversário, jogando corretamente, pode sempre **forçar** uma vitória.

Como exemplo, temos a configuração anterior: 3, 5 e 7, que é uma **Posição insegura**.

Para criar uma posição segura, removemos 3 objetos da fileira de 7 (transformando-a em 4 objetos):

Figura 6: Exemplo de nova configuração.



Agora o novo XOR entre 3, 5 e 4:

Figura 7: XOR 3 e 5.

Bit de 3	Bit de 5	XOR
0	1	1
1	0	1
1	1	0

Figura 8: XOR 6 e 4.

Bit de 6	Bit de 4	XOR
1	1	0
1	0	1
0	0	0

Resultado: 010

3.4 Variáveis que Influenciam o Jogo:

3.4.1 1- Número de Fileiras:

Mais fileiras aumentam as combinações possíveis, dificultando a escolha de uma jogada ideal.

3.4.2 2- Ordem dos Jogadores:

A ordem dos jogadores é crítica porque determina quem tem a última jogada.

Jogar primeiro pode ser uma vantagem se o jogador souber manipular as posições para forçar o adversário a perder.

3.5 Outra Estratégia

Nas tentativas de achar uma lógica para o jogo, percebeu-se que se eu igualasse as quantidades de objetos em cada fileira, ficava mais fácil de vencer. Em quase todas as vezes em que essa estratégia foi utilizada, houve vitória.

Exemplo:

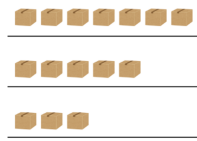
Regras: Quem pegar a última caixa perde o jogo.

Jogadores:

Jogador 1: Humano

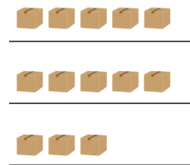
Jogador 2: Máquina

Figura 9: Configuração Inicial.



1ª Jogada (Jogador 2): 2 caixas da fileira de 7.

Figura 10: 1ª Jogada.



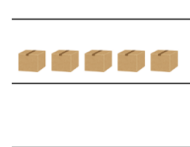
2ª Jogada (Jogador 1): 3 caixas da fileira de 3.

Figura 11: 2ª Jogada.



3ª Jogada (Jogador 2): 5 caixas da fileira de 5.

Figura 12: 3ª Jogada.



4ª Jogada (Jogador 1): 4 caixas da fileira de 5.

Figura 13: 4ª Jogada.



O jogador 2 perdeu, pois foi obrigado a pegar o último palito.

Ao jogar utilizando essa estratégia, não importava quem iniciava as jogadas, pois, no exemplo acima, o jogador 2 começou e, mesmo assim, houve vitória.

O único objetivo era igualar as fileiras com as mesmas quantidades, forçando o oponente a desequilibrá-las e, ao final das jogadas, tudo se resumia à estratégia e à execução do movimento correto com as fileiras.

3.6 Jogo de Tabuleiro

Esse jogo se assemelha ao xadrez, mas de uma forma mais simples, jogado em um tabuleiro 3x3. Nele, dois jogadores competem para alcançar uma das condições de vitória. Ele é usado (neste contexto) para ensinar como máquinas aprendem por meio de tentativas e erros.

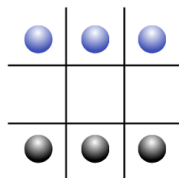
3.6.1 Como Funciona?

O jogo é disputado entre dois jogadores. Cada jogador possui 3 peças, que começam alinhadas em suas fileiras específicas inicialmente.

- Tabuleiro 3x3.
- As peças do jogador 1 (azuis) ocupam a linha superior do tabuleiro.
- As peças do jogador 2 (pretas) ocupam a linha inferior do tabuleiro.

Exemplo:

Figura 14: Jogo do Tabuleiro.



3.6.2 Objetivo

Vencer pelas condições:

- Uma peça atinge a última fileira do adversário (linha oposta).
- Capturar todas as peças do adversário.
- Deixar o adversário sem movimentos possíveis.

3.6.3 Regras Gerais

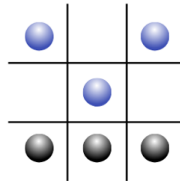
Movimento Válidos:

- Uma peça pode avançar para frente (se a casa estiver vazia).

- Uma peça pode capturar uma peça adversária movendo-se na diagonal para ocupar a casa dela.

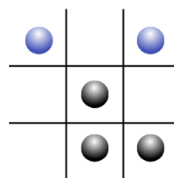
Exemplo:

Figura 15: Jogada 1.



O jogador Preto pode capturar a peça Azul movendo a peça preta para a posição onde está o azul central:

Figura 16: Jogada 2.



- Quando uma peça é capturada, ela é retirada do tabuleiro.
- O jogo termina quando uma das condições de vitória é alcançada.

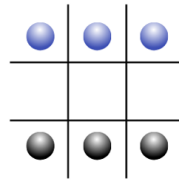
3.6.4 Como o Computador Aprende?

O computador toma decisões baseadas em tentativa e erro, reforçando as boas escolhas e decisões e eliminando as ruins.

- Cada caixa de fósforo representa uma posição do tabuleiro.
- Dentro de cada caixa, há canudos coloridos. Cada cor representa um movimento possível a partir daquela posição.
- A cada turno, um desses movimentos é feito (agita-se a caixa escolhida e seleciona um canudo aleatoriamente).
- A cor do canudo retirado determina o movimento.
- Ao vencer, é adicionado canudos extras para os movimentos que foram realizados nessa partida, reforçando as escolhas.
- Ao perder, os canudos que correspondem aos movimentos que levaram à derrota são removidos.

Exemplo:

Figura 17: Jogo do Tabuleiro.



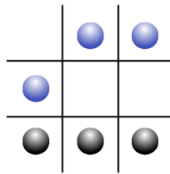
A caixa correspondente ao tabuleiro inicial.

Contém três canudos:

- **Azul:** Mover o B da esquerda para frente.
- **Vermelho:** Mover o B do meio para frente.
- **Verde:** Mover o B da direita para frente.

* Ele sorteia o canudo azul e move a bolinha azul da esquerda para frente:

Figura 18: Jogo do Tabuleiro: Canudo Azul.



- Se o computador vence, ele adiciona um canudo azul extra na caixa inicial.
- Se perde, o canudo azul é removido.

O computador aprende como jogar melhor ao longo de muitas partidas. Este aprendizado funciona como um processo de seleção natural:

- No início, todas as escolhas têm a mesma probabilidade de ocorrer.
- Jogadas ruins (que levam à derrota) são eliminadas, porque os canudos correspondentes são removidos.
- Jogadas boas (que levam à vitória) são reforçadas, porque os canudos são duplicados.

4 Capítulo 4: Máquinas de calcular do arco da velha

Neste capítulo, foram estudadas as diferentes máquinas que realizam operações matemáticas, como soma binária, soma de inteiros e suas variações.

4.1 Somando com água

Para começar, buscou-se compreender o funcionamento dos números binários, ou de base 2.

4.2 Números Binários

Os **binários** são números apenas de **2 algarismos: 0 e 1**. No sistema binário, **cada dígito equivale a 1bit**.

Eles são utilizados pelos PCs para **armazenar informações/dados**. O número "0" representa a **corrente elétrica parada ou desligada**.

Já o número "1" representa a **passagem de corrente elétrica por um circuito ou apenas "ligado"**.

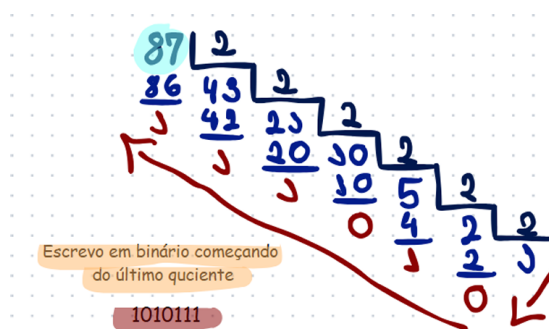
4.2.1 Conversão de número decimal para binário

- 1- Devemos dividir o número que se deseja converter por 2;
- 2- Seguimos dividindo por 2, mas agora, o quociente (resultado da divisão) que é dividido pelo número acima;
- 3- O passo acima é repetido até que o quociente da divisão seja igual a 1;
- 4- E por fim, escreve-se os restos das divisões feitas anteriormente começando com o quociente da última divisão feita.

Exemplo:

O número 87 em binário:

Figura 19: Conversão do número decimal para binário.



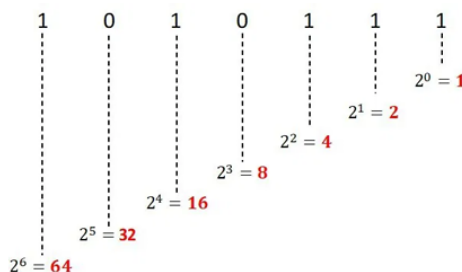
4.2.2 Conversão de número binário para sistema decimal

- 1- Devemos multiplicar cada dígito binário por uma potência de base 2;
- 2- Soma-se os resultados;
- 3- Os expoentes devem ser naturais começando do 0, seguindo em ordem crescente da direita para a esquerda;
- 4- E por último, multiplicamos os bits pelos resultados encontrados nas potências.

Exemplo:

O número 1010111 em decimal:

Figura 20: Conversão de número binário para sistema decimal



Agora, multiplica e soma os resultados:

$$1 \times 1 = 1$$

$$1 \times 2 = 2$$

$$1 \times 4 = 4$$

$$0 \times 8 = 0$$

$$1 \times 16 = 16$$

$$0 \times 32 = 0$$

$$1 \times 64 = 64$$

Somando... Resultado igual a 87.

Após a familiarização com o sistema binário, o foco das pesquisas voltou-se para a compreensão das operações realizadas com esses números.

4.2.3 Operações com números binários

1º) **Adição:**

1) $0 + 0 = 0$

2) $0 + 1 = 1$

3) $1 + 0 = 1$

EXCEÇÃO: $1 + 1 = 10$

A soma entre esses dois bits gera algo diferente na base dois.

Decimal: 2;

Binário: 10 (0 vai um).

Exemplo:

Adicione o número 11 ao 101:

Figura 21: Soma entre binários.

$$\begin{array}{r}
 \\
 \\
 + \\
 \hline
 1 0 0
 \end{array}$$

2º **Subtração:**

1) $1 - 1 = 0$

2) $1 - 0 = 1$

3) $0 - 0 = 0$

EXCEÇÃO: $0 - 1 = 1$

Exemplo:

Subtraia 101 do número 1011:

Figura 22: Subtração entre binários.

$$\begin{array}{r}
 \\
 \\
 - \\
 \hline
 0 1 0
 \end{array}$$

4.2.4 Circuitos Somadores $\rightarrow$ Half Adder

Também foram estudadas as **portas lógicas**. Elas se relacionam com o conceito demonstrado na adição de *números binários*.

Normalmente num circuito somador, utiliza-se apenas 1bit como saída em cada operação de soma binária.

Exemplo:

$1 + 0$ resulta em 1, ou seja, há apenas uma saída. No entanto, como já vimos, existe uma exceção na soma entre $1 + 1$ que na base 2, resulta em 0 com um "vai 1".

Nesse caso, teremos duas saídas:

O bit da soma e o bit de transporte (carry).

4.2.5 Tabela Verdade em um Meio Somador

1º) Duas entradas e uma saída:

Figura 23: Tabela verdade de porta XOR.

Entrada A	Entrada B	Soma (S)
0	0	0
0	1	1
1	0	1
1	1	0 vai 1 (0)

Porta XOR:

- 1 $\rightarrow$ Quando os valores são diferentes

- 0 $\rightarrow$ Quando os valores são iguais

Como já visto, ao somar $1 + 1$ o resultado é 0 com "vai 1".

Nesse caso, precisamos de mais um pino de na saída.

2º) Duas entradas e duas saídas:

Figura 24: Tabela verdade de porta AND.

Entrada A	Entrada B	Soma (S)	Vai 1 (C) $\rightarrow$ carry
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	vai 1

Agora a tabela verdade tem 2 saídas. O carry é representado por $A \text{ AND } B$, pois o seu valor é 1 apenas quando tanto A quanto B são iguais a 1.

→ Agora há duas saídas:

- **SAÍDA S:**

- 2 saídas validas 1 e 1

- Porta XOR

- $S = A \text{ XOR } B$

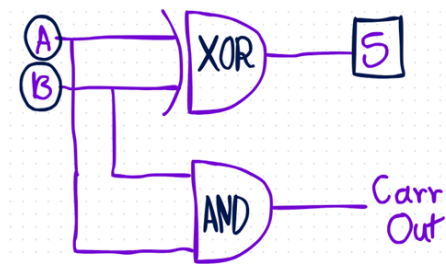
- **SAÍDA C:**

- 1 saída valida 1

- Porta AND

- $C = A \text{ AND } B$

Figura 25: Circuito porta AND.



4.3 Calculadora do Fred Flintstone

É uma máquina que opera como calculadora manual por meio de colunas. Há 3 retas:

Duas das extremidades: são as parcelas; Reta central: resultado da adição.

Exemplo:

Figura 26: Soma de Fred.

3	6	3
2	4	2
1	2	1

Essa máquina, assim como as próximas, são uma forma simplificada do que o computador faz internamente.

4.4 Lata de Calcular

Utiliza-se réguas circulares pra realizar as operações com número inteiros, junto com uma tampa giratória.

Para testá-la, foi utilizada uma garrafa d'água, na qual foram montadas duas retas numéricas que vão de -10 a 10, uma fixada na tampa e outra no corpo da garrafa.

Os zeros de ambas as retas foram alinhados para iniciar os testes.

Primeira Tentativa: $(-2) + 1$

O número -2 estava marcado na régua da tampa, e o resultado apareceu na régua do corpo.

Como se tratava de uma soma, a tampa foi girada uma unidade para a direita.

O -2 da tampa se alinhou com o -1 do corpo, indicando o resultado: $-2 + 1 = -1$.

Segunda Tentativa: $(-2) - 3$

Para isso, a tampa foi girada três unidades para a esquerda, pois se trata de uma subtração.

O -2 da tampa se alinhou com o -5 do corpo da garrafa, indicando que o resultado: $-2 - 3 = -5$.

4.5 A Calculadora Parabólica

Ela permite calcular o produto entre dois números.

Basta marcar dois valores, **a** e **b**, sobre o **eixo x** de um plano em que está representada a parábola. Em seguida, estica-se uma corda ligando os pontos correspondentes a **a** e **b** na parábola. O ponto em que essa corda intercepta o **eixo y** representa o produto de **a** $\times$ **b**.

4.6 Pelas Barbas de Naipier

É um método de multiplicação através de barras. Ele utiliza uma tabela de multiplicar com partes móveis, composta por barras numeradas.

Cada barra representa um algarismo e é dividida em 9 quadrados, c correspondentes aos números de 1 a 9.

Dentro de cada quadrado, há dois triângulos:

1- Triângulo superior: Mostra as dezenas do produto.

2- Triângulo inferior: Mostra as unidades do produto.

Ao alinhar as barras correspondentes aos algarismos de um número com os valores de outro número (de 1 a 9), é possível visualizar e somar os produtos para encontrar o resultado final da multiplicação.

Exemplo:

1) $3 \times 2 = 6$

2) $5 \times 6 = 30$

3) $8 \times 9 = 72$

Figura 27: Pelas Barbas de Naipier.

	0	1	2	3	4	5	6	7	8	9	
1	/	1	2	3	4	5	6	7	8	9	1
2	/	2	4	6	8	10	12	14	16	18	2
3	/	3	6	9	12	15	18	21	24	27	3
4	/	4	8	12	16	20	24	28	32	36	4
5	/	5	10	15	20	25	30	35	40	45	5
6	/	6	12	18	24	30	36	42	48	54	6
7	/	7	14	21	28	35	42	49	56	63	7
8	/	8	16	24	32	40	48	56	64	72	8
9	/	9	18	27	36	45	54	63	72	81	9

4.7 A Origem dos Computadores

O foco dessa parte consistiu em compreender como os computadores surgiram e como se desenvolveram ao longo da história.

O primeiro registo de instrumento de cálculo é o ábaco. Depois dele vários outros instrumentos surgiram.

Além disso, cientistas inventaram as linguagens de programação.

As tecnologias antigas evoluíram e serviram de base pras atuais.

4.8 Funcionamento dos Computadores Analógicos

Por que usar macarrões pra entender algo que é óbvio e que não necessita de lógica, apenas de fatos? Essa foi a pergunta levantada durante o estudo desse capítulo.

Entendeu-se que se tratava apenas de uma demonstração prática de como funciona o raciocínio dos computadores analógicos, mostrando que, antes dos PCs digitais, os analógicos utilizavam propriedades físicas para resolver problemas.

O exemplo do macarrão simula um computador analógico

Exemplo:

Deseja-se ordenar uma lista de números naturais em ordem decrescente: 7, 12, 4, 18, 11, 10, 21, 9, 13

Em vez de usar um computador digital que faz isso com códigos, você pode usar um método analógico, com varetas de espaguete cru como se fossem os números.

Como Funciona?

1. Pegue uma vareta no comprimento dos números.
2. Junte as varetas num feixe, de pé, com as pontas na mesa.
3. Como os tamanhos são diferentes, a vareta mais longa vai se destacar pra cima.
4. Basta ir puxando as varetas de cima pra baixo sempre pegando a mais comprida que sobrou.

Nesse caso, a 1ª a ser retirada do feixe é a de tamanho 21cm que corresponde ao número 21.

Exemplo do Barbante:

Neste caso, deseja-se saber qual caminho é o mais curto entre uma cidade e outra.

Basta usar um barbante com vários nós. Cada nó representa uma cidade. O comprimento entre dois nós representa a distância da estrada entre duas cidades. O barbante está todo bagunçado, como uma teia com conexões entre várias cidades.

1. Segure o nó da cidade de origem com a mão.
2. Segure o nó da cidade destino na outra.
3. Estique o barbante.

Automaticamente, o fio mais esticado e reto vai mostrar o caminho mais curto entre os dois pontos, porque a menor distância física entre os nós será a menor estrada possível dentro daquela teia de caminhos.

5 Capítulo 5: Conversando com Máquinas

5.1 A Teoria da Caixa Preta

A caixa preta tem três botões e uma lâmpada:

1- Preto

2- Vermelho

3- Amarelo

→ **Caixa:** máquina desconhecida;

→ **Botão Preto:** Reset, reinicia e apaga a lâmpada;

→ **Botão Vermelho:** Letra A;

→ **Botão Amarelo:** Letra B.

5.2 Objetivo

O objetivo desta atividade é tentar descobrir a linguagem usada pela máquina com base em tentativa, erro e lógica. É necessário testar sequências de botões para entender quais palavras são aceitas.

5.3 O Conceito de Autômato

Nas pesquisas, foi descoberto o significado de “autômatos” — um modelo teórico de computador simples, usado para estudar linguagens formais, que são conjuntos de símbolos organizados por regras rígidas (gramática).

5.4 Lógica da Máquina

Ao longo dos testes, percebeu-se que a máquina ignora os **b's** e entende apenas quando há um número ímpar de letras **a's** (considerando que o estado inicial da lâmpada está apagada).

→ **Estado da lâmpada:**

1- Apagada: estado inicial.

2- Acesa: estado final/desejado.

5.4.1 Como Cada Letra Afeta o Estado da Lâmpada

a: Inverte o estado (apaga ou acende).

b: Não muda o estado da lâmpada.

Exemplo:

1) Entradas: a

- Começa apagada
- "a"acende a lâmpada

→ **Palavra aceita.**

2) Entradas: aa

- Começa apagada
- "a"acende a lâmpada
- "a"acende a lâmpada

→ **Palavra não aceita.**

3) bab

- Começa apagada
- "b"mantém a lâmpada apagada
- "a"acende a lâmpada
- "b"mantém a lâmpada acesa

→ **Palavra aceita.**

A máquina funciona como um contador de letras **a's** e se o número delas for ímpar, a lâmpada termina acesa; e se for par, a lâmpada termina apagada.

O estado inicial da lâmpada interfere nessa contagem. Se o número de **a's** forem ímpar, a lâmpada se mantém apagada e se o número for par, a lâmpada se acende.

5.4.2 Outras Máquinas e Outras Linguagens

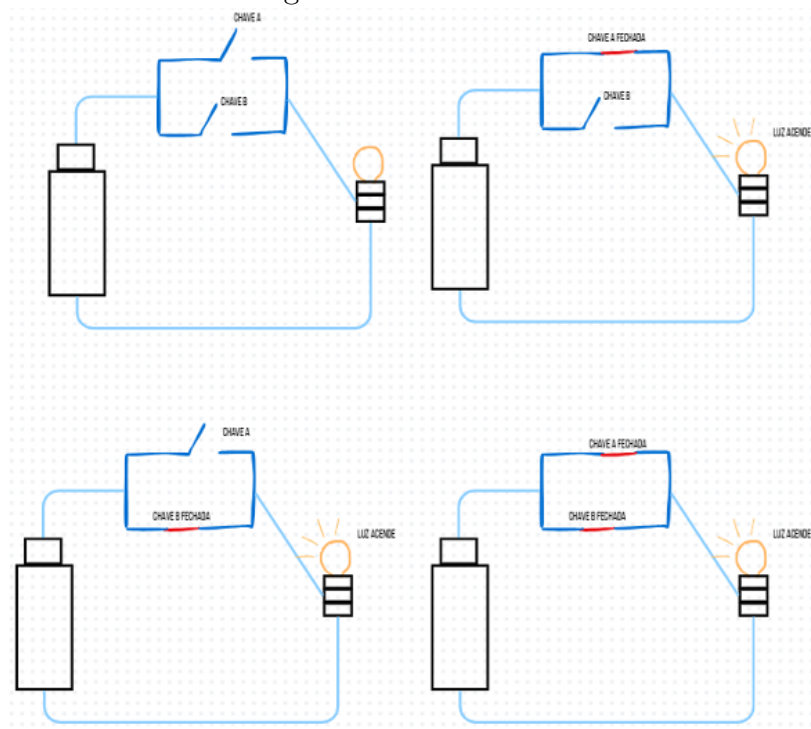
→ **Máquina OU:**

A lâmpada só acende se pelo menos uma das chaves forem fechadas ou as duas ao mesmo tempo.

Figura 28: Tabela OU.

p	q	$p \vee q$
v	v	v
v	f	v
f	v	v
f	f	f

Figura 29: Circuito OU.



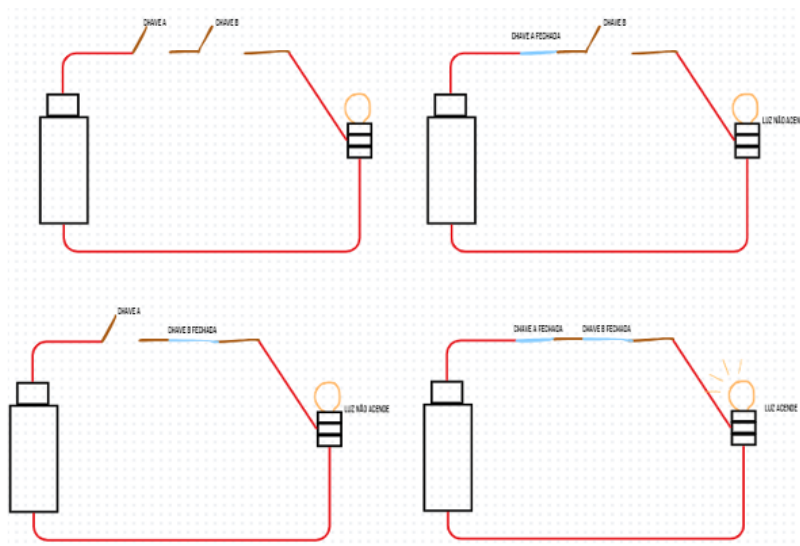
→ **Máquina E:**

Neste caso, para que a luz acenda, é necessário que as duas chaves estejam fechadas.

Figura 30: Tabela E.

p	q	$p \wedge q$
v	v	v
v	f	f
f	v	f
f	f	f

Figura 31: Circuito E.



6 Capítulo 6: Autômatos

Como dito anteriormente, os autômatos são máquinas que leem palavras (sequências de símbolos) e decidem se elas fazem parte de uma linguagem de acordo com as regras pré-estabelecidas. É composta por estados possíveis e regras de transição de um estado pro outro.

Foi encontrado um exemplo que auxiliou na compreensão do assunto:

Pense numa porta automática que se abre com um sensor de presença.

Dois estados: aberta e fechada

Condições:

- Se a porta está fechada e não há ninguém perto: continua fechada
- Se a porta está fechada, mas há alguém perto: a porta abre
- Se estiver aberta, mas alguém perto: ela se fecha
- Se estiver aberta com alguém perto: se mantém aberta

6.1 Sistema de Estados Finitos e Determinísticos

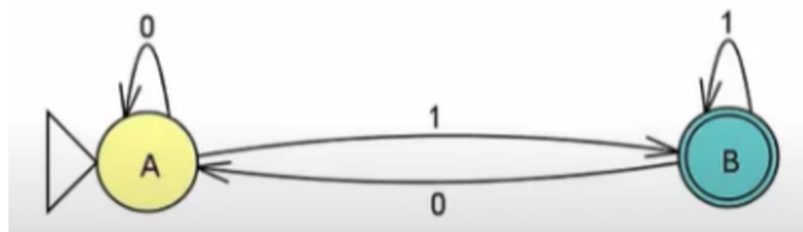
Pesquisando outras fontes, descobriu-se que um autômato finito pode ser definido como uma quintupla $(S, w, \hat{o}, s_0, F)$:

- **s**: conjunto finito dos estados positivos
- **w**: conjunto que constitui o alfabeto dos autômatos
- **$\hat{o}$** : função de transição ($\hat{o}(q_0, a) = q_1 \rightarrow$ se estou em q_0 e leio “a”, vou para q_1)
- $\hat{o}(q_1, b) = q_1 \rightarrow$ se estou em q_1 e leio “b”, continuo em q_1)
- **s_0** : estado inicial

OBS: A função $\hat{o}$ determina o comportamento da máquina e as transições de estado para cada símbolo do alfabeto

- **F**: conjunto de estados finais

Figura 32: Exemplo.



Entradas: 0 e 1

- Se o estado atual é A e a entrada é 0, permanece-se no estado A.
- Se o estado atual é A e a entrada é 1, vai-se para o estado B.
- Se o estado atual é B e a entrada é 1, permanece-se no estado B.
- Se o estado atual é B e a entrada é 0, vai-se para o estado A (Considere a cadeia 110).
- **1**: vai-se para B.
- **1**: permanece-se em B.
- **0**: retorna-se para A.

Essa cadeia é rejeitada, pois **A** não é o estado final.

Todo autômato segue um alfabeto e um alfabeto é um conjunto de símbolos finitos.

Exemplo:

$w = 0,1 \rightarrow$ Sigma Binário de 2 elementos

Através desse alfabeto, é possível formar uma cadeia sobre ele. Cadeias são sequências de símbolos retirados do alfabeto.

Exemplo:

$w = a,b$

“a”, “b”, “ab”, “ba”, “aab”, “abba” são cadeias

“abc”, “ax” não são cadeias válidas dentro deste alfabeto

Para saber o comprimento de uma cadeia, basta analisar a quantidade de símbolos que ela tem/comprimento.

Exemplo:

$m = 011011$ é cadeia sobre $w = 0,1$ e $|m| = 6$

Exemplo de linguagem:

Figura 33: Exemplo de Linguagem.

$$\begin{aligned}
 L_1 &= \{ w \in \{0,1\}^* : |w|_0 = |w|_1 \} = \{ 01, 10, 11101000, \epsilon, \dots \} \\
 L_2 &= \{ w \in \{a,b\}^* : |w|_a \leq |w|_b \} = \{ b, \epsilon, aba, aaaa, \dots \} \\
 L_3 &= \{ 0^m 1^n : m \geq 1 \} = \{ 01, 0011, 000111, \dots \} \\
 L_4 &= \{ a^i b^j : 0 \leq i \leq j \} = \{ \epsilon, ab, abbb, aabbbb, ab, bbb, \dots \} \\
 L_5 &= \{ \epsilon \} \\
 L_6 &= \{ \} = \emptyset \\
 L_7 &= \{ w \in \{0,1,2\}^* : \text{5º símbolo de } w \text{ é } 2 \} = \{ 012121, \dots \}
 \end{aligned}$$

6.2 Lema do Bombardeio

Ele prova se uma linguagem é regular ou não.

Linguagem Regular ou Autômato Finito: linguagem que pode ser entendida por uma máquina simples. Ela não possui memória, apenas vai de um estado para o outro de acordo com as letras das palavras lidas.

Exemplo:

Palavras: aaabbb

Autômato:

$a \rightarrow$ vai pra q1

$a \rightarrow$ vai pra q2

$a \rightarrow$ vai pra q3

$b \rightarrow$ vai pra q_4

Uma linguagem só é regular se tiver palavras muito grandes em que partes delas podem ser repetidas várias vezes e ainda assim, essa nova palavra continua pertencendo à linguagem.

Esse modelo de autômato não é muito útil para linguagens que exigem contas, como as que precisam ter o mesmo número de de "a"s e "b"s.

7 Capítulo 7: Como uma Máquina Pode se Lembrar

Aqui foi estudado como um computador utiliza a memória para compreender algumas linguagens.

Considere um alfabeto **a,b** com o qual podem ser formadas várias palavras, como: abba, abab, aaa, entre outras. Todas essas combinações formam um conjunto **A**. A partir dele, podem ser selecionadas apenas algumas palavras específicas — por exemplo, as que começam e terminam com ‘**a**’ — formando uma linguagem, isto é, um **subconjunto de palavras que obedecem a uma determinada regra**.

7.1 Máquinas que reconhecem linguagens

Algumas máquinas conseguem dizer se uma palavra pertence ou não a uma linguagem. Essas máquinas são chamadas de autômatos finitos. São como robôs com memória bem limitada. Mas algumas linguagens não são tão simples, como a dos palíndromos, que são palavras que são iguais quando lidas de trás pra frente, como "**ovo**" ou "**radar**".

Os **Autômatos Finitos** não têm memória pra conferir isso. Ela não guarda o início da palavra pra comparar depois.

O que é uma pilha?

São como pilhas de pratos: você só consegue colocar ou tirar pratos pelo topo da pilha. Isso é uma estrutura de memória.

7.2 O Autômato de Pilha

Esse autômato é mais inteligente e eficiente que o '*finito*', porque ele tem a pilha de memória. Basicamente, ela é uma máquina que "*lembra*" palíndromas.

Exemplo:

Alfabeto: a,b,c

A palavra tem 3 partes:

p c p'

p: primeira parte da palavra(ex: "ab")

c: um separador no meio

p': é a parte p de trás pra frente(ex: "ba")

Exemplo: "abcba"

- A pilha começa com um prato vermelho (demarca o fundo da pilha).
- A máquina está no estado q1.
- A máquina começa lendo a parte p.
- Se ela ler a, empilha um prato azul.
- Se ler b, empilha um prato verde.

Então, a máquina segue guardando/empilhando como se estivesse memorizando a primeira parte da palavra.

Exemplo: "abbc..."

azul (a)

verde (b)

verde (b)

Quando a máquina lê o símbolo c, ela para de empilhar(q1) e entra no estado q2.

Agora, ela vai comparar o está digitando com o que está na pilha de pratos.

Se em algum momento o prato não combinar com o que foi digitado, a palavra é rejeitada.

No final, deve restar apenas o prato vermelho que é retirado automaticamente da pilha (ficando vazia). Se isso acontecer, significa que a palavra foi aceita.

Exemplo: "abbcbbba"

Parte "p":

a → empilha azul

b → empilha verde

b → empilha verde

De cima para baixo teremos:

verde (b)

verde (b)

azul (a)

prato vermelho

Parte p':

$b \rightarrow \text{remove verde}$

$b \rightarrow \text{remove verde}$

$a \rightarrow \text{remove azul}$

Resta apenas o prato vermelho, que é removido, ficando com uma pilha vazia. Portanto, a palavra foi aceita.

OBS* (O último prato que entra é o primeiro que sai).

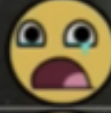
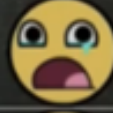
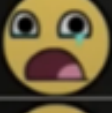
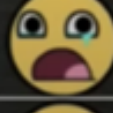
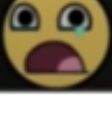
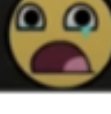
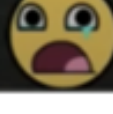
7.3 Memória Eletrônica

As máquinas guardam os dados e "se lembram" depois. Só que no caso das máquinas, essa memória é formada por circuitos.

Pra entender essa parte, usou-se os conceitos anteriores sobre lógica com a tabela verdade.

Tem-se o conectivo E, que pressupõe que uma sentença é verdadeira se e somente se todas forem verdadeiras.

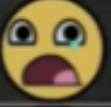
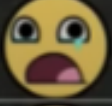
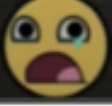
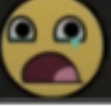
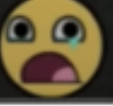
Figura 34: Tabela E.

p	q	$p \text{ E } q$
		
		
		
		

Esse operador só me retorna uma sentença verdadeira quando os dois amigos estão felizes.

Agora, com o operador OU, tem-se que para que uma sentença seja verdadeira, podemos ter apenas um elemento verdadeiro.

Figura 35: Tabela OU.

p	q	p OU q
		
		
		
		

Esse operador só retorna uma sentença verdadeira quando pelo menos um dos amigos está feliz.

Por últimos, tem-se o **NÃO**. Ele funciona trocando os estados dos resultados das sentenças.

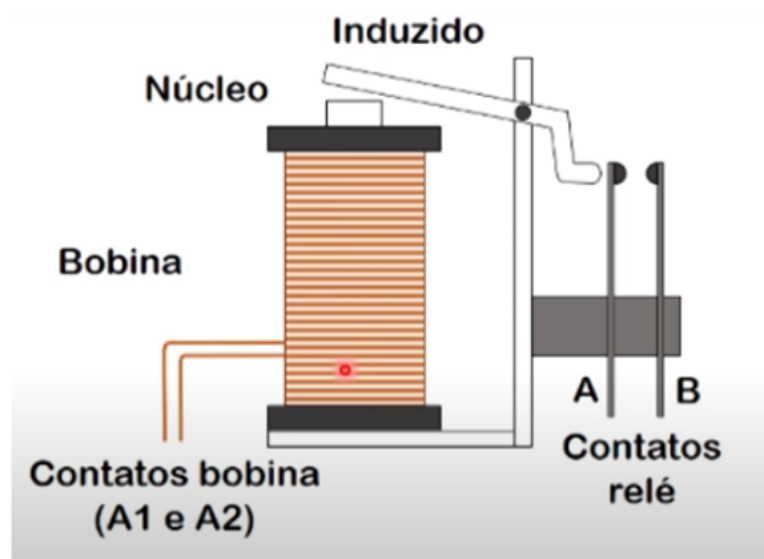
Ex: O que era verdadeiro vira falso e o contrário também é verdadeiro.

Essa lógica também se aplica aos circuitos, mas no caso do operador NÃO, é necessário usar os relês, pois imagine que para que uma lâmpada acenda, é preciso que não haja corrente e que para se apague, é preciso que haja corrente.

7.3.1 Modelo Relê

Como se fosse um interruptor inteligente. Funciona como uma alavanca que muda quando passa energia. Quando não tem corrente, ele mantém a luz ligada e quando passa energia, ele desliga a luz.

Figura 36: Modelo Relê.



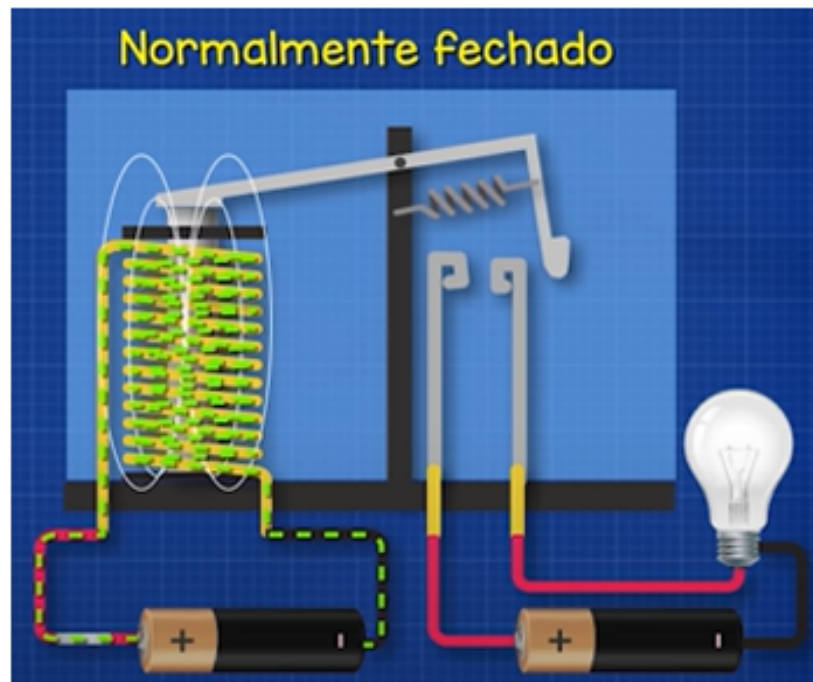
A figura mostra um relê **com parte mecânica**. A parte principal é a **Bobina** de cobre que quando **submetido a uma tensão**, gera um **eletro-ímã**. O **Núcleo** atrai o **Induzido** (parte móvel) que por sua vez, empurra o **contato A** que se fecha com o **contato B**. Teremos então, um **contado fechado**.

Figura 37: Relê Normalmente Fechado.



Na figura 37, quando não há carga, a luz acende.

Figura 38: Relê Normalmente Fechado.



Neste outro caso, quando há passagem de corrente pelo circuito, a lâmpada se apaga.

7.3.2 Flip-Flops

São dispositivos usados pra armazenar 'estados', portanto, ele possui dois estados: **0 e 1**. Eles podem **armazenar** ou **transferir dados**.

Cada *Flip-Flop* possui dois pinos, um de **entrada** e outro de **saída**.

7.3.3 Flip-Flop Set and Reset

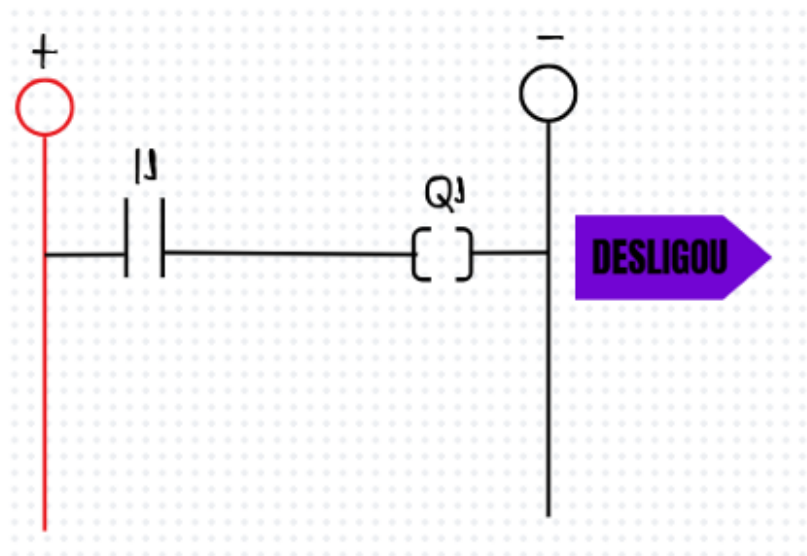
São usados para colocar 1 (corrente) ou colocar 0 ((sem corrente (Set/Reset))). Portanto possuem duas entradas e duas saídas.

Esse modelo é muito útil para os computadores, pois são capazes de armazenar dados mesmo quando as suas entradas são modificadas/desligadas, ao contrários dos circuitos anteriores.

OBS: A saída Q é sempre o inverso de Q. Então se $Q = 1$; $\bar{Q} = 0$.

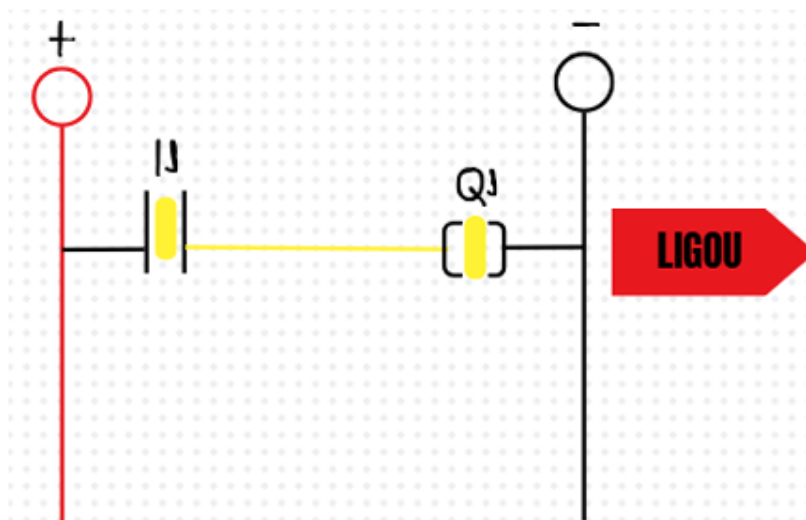
Houve dificuldade na compreensão dessa parte da pesquisa, o que levou à busca por outras fontes de estudo.

Figura 39: Bobina Normal Desligada.



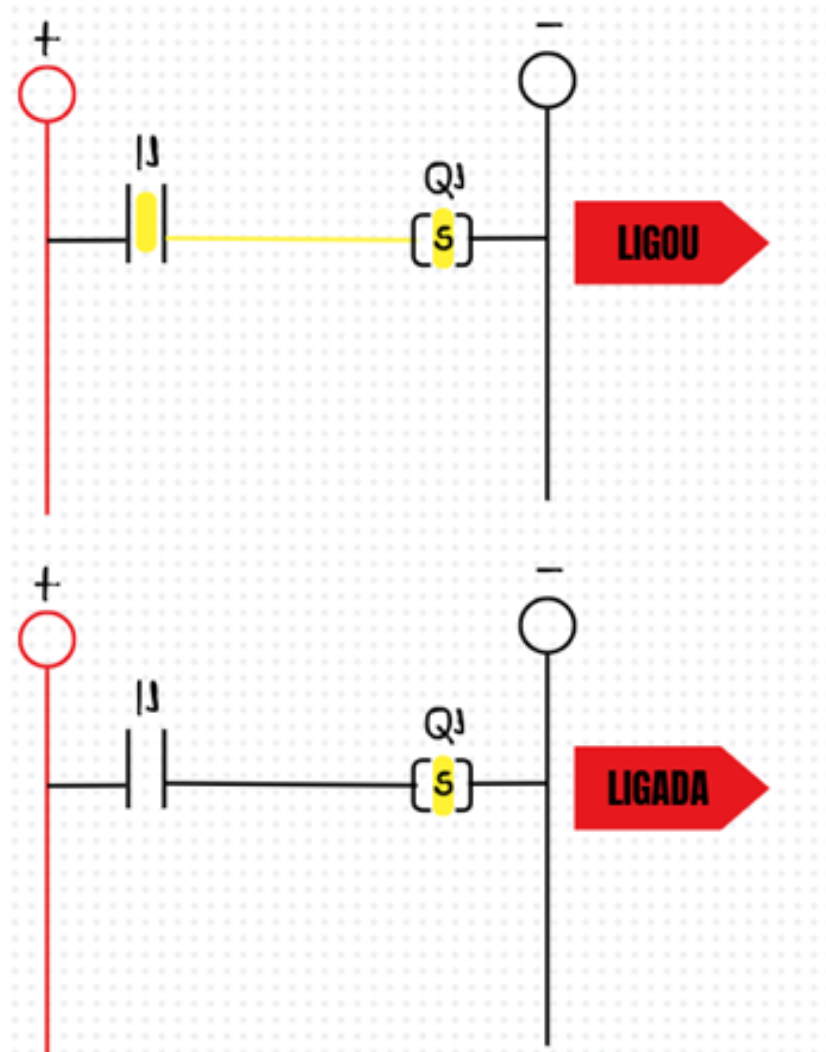
Quando ela recebe energia, uma saída é acionada.

Figura 40: Bobina Normal Ligada.



Quando ela não recebe mais energia, ela desliga, mas se colocarmos um outro tipo de bobina (set), o contrário acontece. Ou seja, ela mantém a bobina ligada mesmo quando não há energia.

Figura 41: Bobina Set.



Mas agora que ela está ligada, como pode desligar já que está setada?

Pode-se usar uma outra bobina: a bobina *reset* (ela desliga o que a bobina set ligou).

Figura 42: Bobina Set Reset.

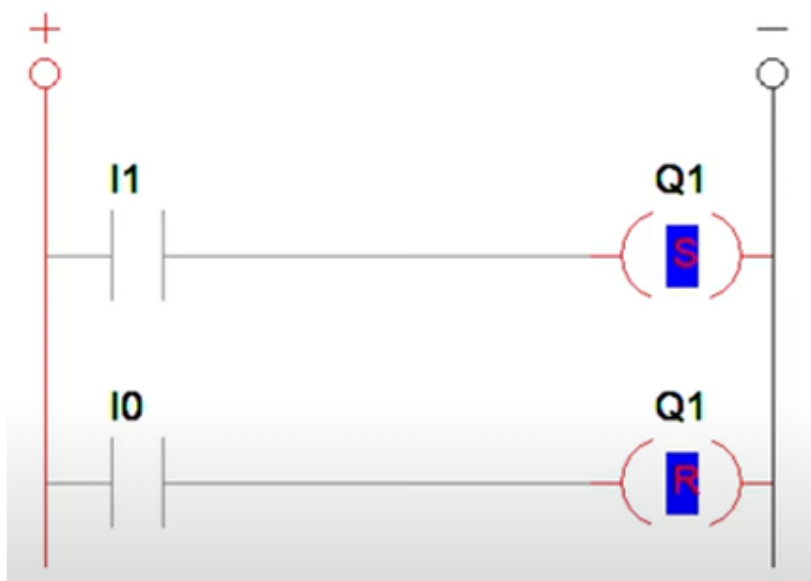


Figura 43: Bobina Set Reset.

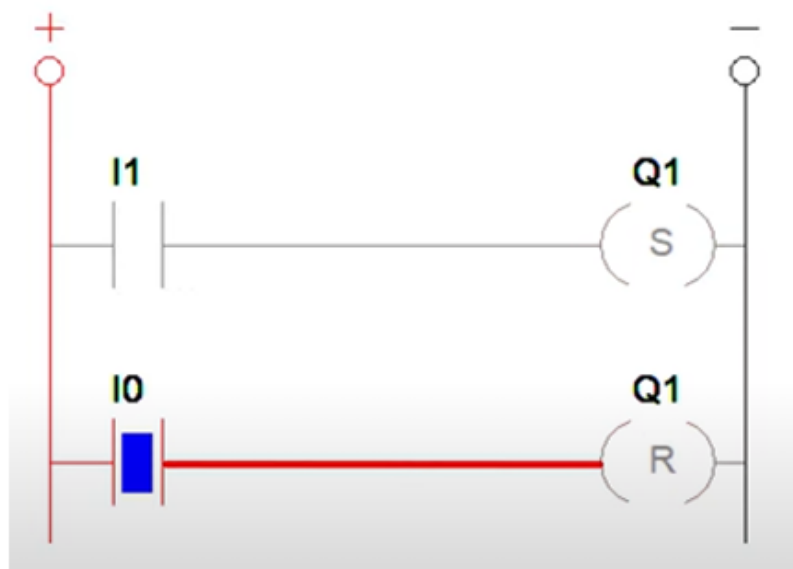


Figura 44: Flip-Flop S/R.

S	R	Q	Q\
0	0	1 ou 0	0 ou 1
0	1	0	1
1	0	1	0
1	1	Inválido	Inválido

1- Quando o Flip-Flop tem 0 e 0 nas suas entradas: Mantém o seu estado inicial (se estava 1, fica 1.)

2- Quando o Flip-Flop tem 1 no Reset: Força o 0 no Q

3- Quando o Flip-Flop tem 1 no Set: Força o 1 no Q

4- Quando o Flip-Flop tem 1 no Set e 1 no Reset: Resultado inválido, pois estamos forçando os dois Qs a serem 1

Exemplo:

Ligar uma luz (set = 1)

- Basta puxar a alavanca $S = 1$ e $R = 0$

A luz acende; $Q = 1$

Exemplo:

Apagar a luz (set = 0)

- Basta puxar a alavanca $R = 1$ e $S = 0$

A luz apaga; $Q = 0$

Para deixar tudo parado, basta deixar tudo parado $S = 0$ e $R = 0$. Se estava 1, continua 1 e se estava 0, continua 0.

Agora imagine que começa com $S = 1$ e $R = 0$, e de repente mudamos o $R = 1$ e o $S = 0$.

1° A lâmpada acende

2° Pedimos pra apagar, mas o S continua sendo 1

O resultado é que nada acontece porque o Flip-Flop tem memória, então ele só vai considerar a entrada anterior. Então mesmo com a entrada mudando, o resultado não.

8 Capítulo 8: Gramática Combina com Matemática?

A **gramática** é o conjunto de regras que validam e organizam palavras pra formarem frases com sentido.

As **máquinas** também precisam de regras pra entender comando.

Pra que um computador entenda uma sentença, deve-se usar **classificações específicas**. Usa-se as produções, que são regras pra transformar uma *<frase>* num *<sujeito>* *<predicado>* por exemplo.

A Gramática é formada por:

1- Terminais: Palavras finais (Ex: letras 'a' e 'b')

2- Variáveis: Representam grupos (Ex: *<verbo>* e *<sujeito>*)

3- Produções: Regras que transforma as variáveis em outras classificações (Ex: *<sujeito>* -> *<nucleo>* *<adjunto>*)

4- Símbolo inicial: Começo da formação (Ex: *<sentença>*)

8.1 Procedimentos VS Algoritmos

Os procedimentos são como seguir receitas. Um programa segue o passo a passo de um procedimento.

Se o procedimento fornece um resultado, então ele é considerado um algoritmo.

Já os algoritmos são os procedimentos que sempre terminam fornecendo uma resposta.

9 Capítulo 9: O Mais Poderoso de Todos os Computadores

9.1 Máquina de Turing

A máquina de Turing funciona como uma longa fita dividida em quadrados, onde cada um pode conter um 0, um 1 ou estar vazio.

Ela analisa essa fita um quadrado por vez e, dependendo do símbolo lido e do estado em que se encontra, realiza uma ação.

Por exemplo, ao estar no estado 23 e ler um 0, a máquina pode apagar o 0, escrever um 1, mover a cabeça de leitura para a direita e mudar para o estado 24.

Depois, aparece a parte do jogo com símbolos e cores. Trata-se de um mini tabuleiro 3x3 com quadrados numerados.

Em cada quadrado, há uma figura colorida, como triângulo azul, círculo amarelo, entre outros.

Ao lado, há uma lista de regras que indicam quando é necessário trocar a cor ou o formato das figuras.

Nas instruções, aparece um código como "711", que significa: aplicar a regra 7 no quadrado número 11 do tabuleiro.

9.2 RAM- Máquina com Acesso Aleatório

A Máquina de Turing é lenta, pois precisa analisar bloco por bloco. Mesmo sendo tão lenta, ela inspirou a base computacional que usamos hoje

Atualmente, os computadores são mais rápidos, pois podem acessar qualquer parte da memória sem precisar passar por todos os endereços.

Além disso, foi descoberto que existe um **acumulador**, onde os valores são guardados temporariamente (como uma memória de trabalho). Os **endereços** são onde guardamos os números e a **unidade de controle**, em que se lê as instruções e controla tudo.

10 Capítulo 10: A Vida Sexual das Máquinas

10.1 Autômatos Celulares

Nesta parte das pesquisas, foi estudado sobre os autômatos celulares, que são máquinas abstratas, ou seja, um funcionamento de máquina, mas não é algo físico. Seu funcionamento é feito por regras baseadas pela vizinhança. O seu espaço é dividido em mini células (quadrados).

10.2 Jogo da Vida

Regras:

- 1- Cada posição do espaço tem um estado: Vivo ou morto.
- 2- Se uma célula tem menos de 2 vizinhos vivos, numa próxima geração, essa célula morre.
- 3- Se essa célula tem 2 ou 3 vizinhos vivos, ela vive.
- 4- Se ela tiver mais de 3, ela morre.
- 5- E se uma célula morta, tiver exatamente 3 vizinhos, ela nasce.

Exemplo:

Tem-se duas grades sendo que a 1° representa a geração atual.

Cada célula que a compões será analisada para a próxima geração.

Figura 45: 1° Geração.

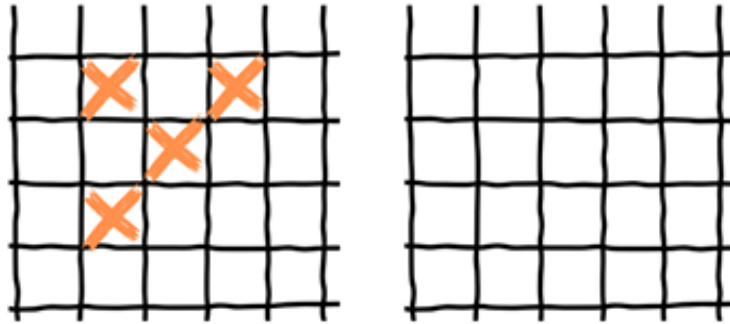
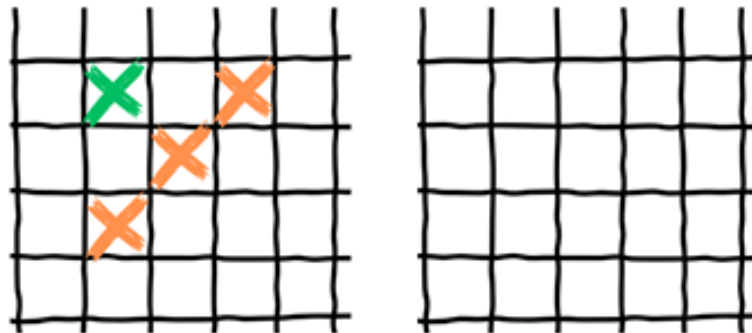
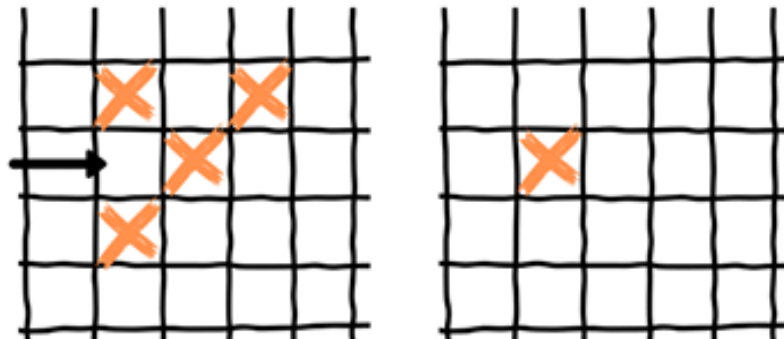


Figura 46: 2° Geração.



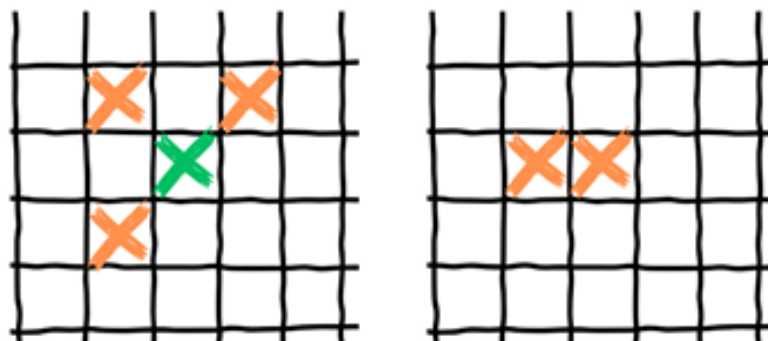
A célula destacada em verde morre na próxima geração por ter apenas um vizinho.

Figura 47: 3° Geração.



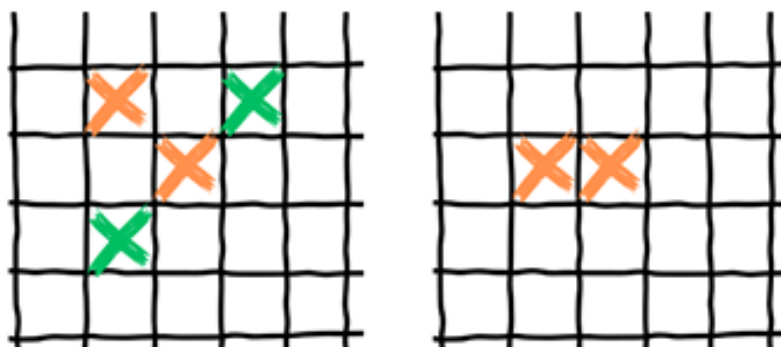
A célula destacada pela seta nasce na próxima geração por ter três vizinhos.

Figura 48: 4° Geração.



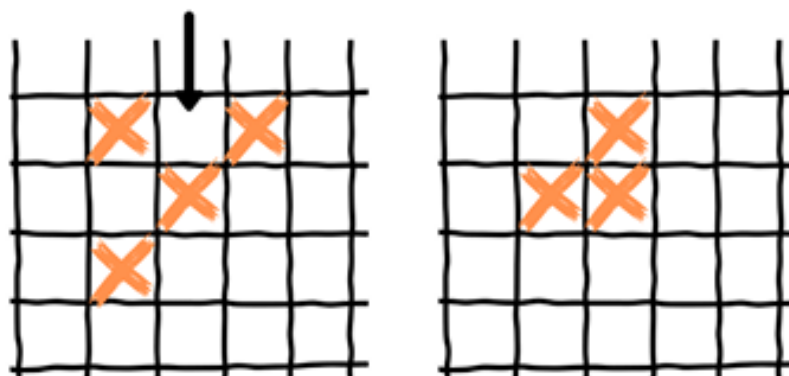
A célula em verde permanece viva na próxima geração.

Figura 49: 5° Geração.



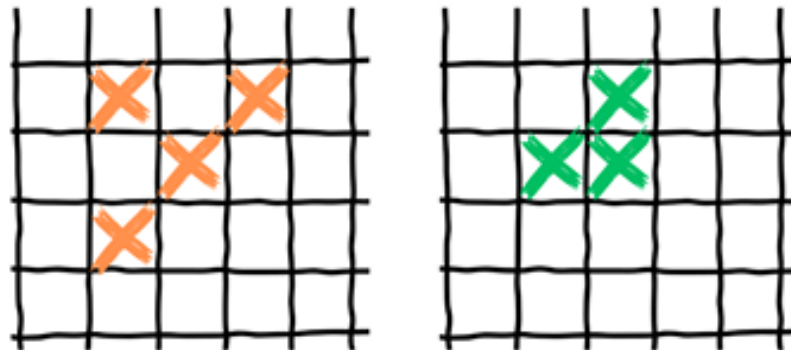
Essas duas também morrem na próxima geração.

Figura 50: 6° Geração.



A célula destacada pela seta nasce na próxima geração por ter três vizinhos.

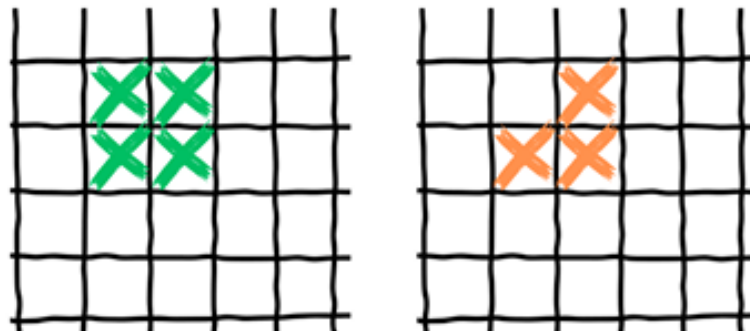
Figura 51: Configuração Final.



Essa é a configuração final do jogo da vida.

E o jogo segue até chegar nesta configuração em verde que não se transforma mais.

Figura 52: Nova Configuração.



11 Capítulo 11: O que a Máquina Faz, Mas Não Deveria Fazer

Houve dificuldade em compreender a linguagem abordada pelo livro neste capítulo e, por isso, foram buscadas novas fontes de estudo.

Algoritmo é uma sequência de passos lógicos que resolvem um problema.

Exemplo:

Figura 53: Exemplo de Algoritmo.

```
Java ▾  
  
public static int somar(int a, int b) {  
    return a + b;  
}  
  
// soma entre dois números|
```

Nem todos os algoritmos são iguais. Às vezes, dois algoritmos resolvem o mesmo problema, mas de formas diferentes.

Por isso, eles são avaliados pra saber qual é melhor.

Para que eles sejam bons, precisam resolver problemas corretamente, resolver de forma rápida e eficiente, usar uma quantidade razoável de memória e funcionar bem mesmo com entradas de dados muito grandes.

O tempo de execução está relacionado não aos segundos, mas em quantidade de operações que ele faz dependendo do tamanho da entrada.

Cada operação conta como 1 passo e o tempo de execução como o número total desses passos.

Exemplo:

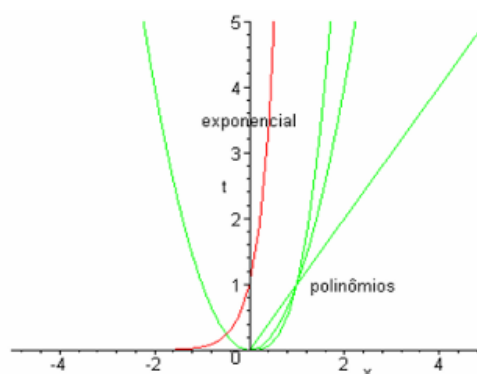
Um algoritmo que soma elementos de uma lista.

Se a lista tem 10 números, leva um tempo.

Se tem 100 números, leva mais tempo.

Então o tempo depende do tamanho da entrada. O tamanho da entrada é representado pela letra n .

Figura 54: Gráfico.



Nesse gráfico mostra como o tempo de execução cresce quando n aumenta. Quanto mais curva pra cima, mais lento é o algoritmo. Os piores são exponenciais e fatoriais.

11.1 Classes P e NP

Classe P

São problemas que podem ser resolvidos rápido.

Exemplo: somar, ordenar, multiplicar números.

Classe NP

Problemas que são difíceis de resolver, mas se alguém te der a resposta, você consegue verificar rápido.

Exemplo: Sudoku, quebras de senhas, combinações.

A pergunta é:

“Se eu consigo verificar uma resposta rapidamente (NP), será que existe algum jeito de resolver o problema também rapidamente (P)?”

11.2 Máquina de Turing

É um modelo teórico de computador. Ela é usada pra entender se um problema podem ser resolvidos por algoritmos ou não.

11.2.1 Determinística

Em uma situação, a máquina só tem um caminho apenas. Como um algoritmo que segue os mesmos passos e escolhas.

A classe **P** pode ser resolvida por uma máquina determinística.

Exemplo: Ordenar uma lista, multiplicar números.

11.2.2 Não-Determinísticos

A máquina pode escolher vários caminhos ao mesmo tempo. Ela não existe fisicamente, mas é usada como um modelo teórico.

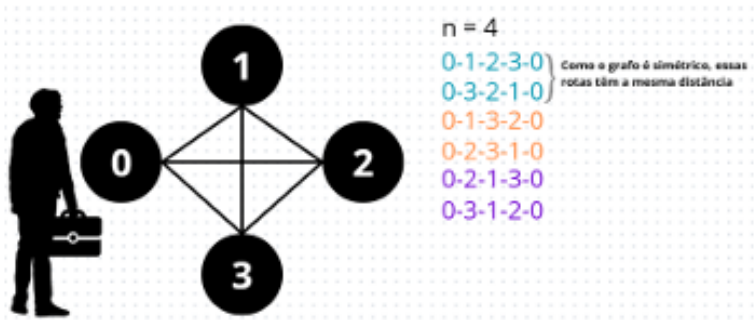
Há uma classe dentro da **NP** que são os mais difíceis e se forem resolvidos, qualquer problema dentro da classe principal pode ser resolvidos também. Se um problema **NP-completo** for resolvido rapidamente significa que $P = NP$.

Exemplo: Problema do Caixeiro Viajante (**TSP**)

Essa é a história de um caixeiro que planejou uma viagem de trabalho. Ele vai visitar um total de **n cidades** saindo da casa dele e visita **n-1** cidades e no final do dia ele volta pra casa.

O objetivo do problema é encontrar a rota de menor distância.

Figura 55: Grafo.



$$(n - 1)! \div 2$$

Isso porque ele sempre começa na mesma cidade $\rightarrow$ fixamos ela. Depois, permutamos **n-1** cidades restantes.

Dividimos por dois porque o grafo é simétrico então basta inverter um caminho.

$$n!$$

$$4! = 4.3.2.1 = 24$$

O número de possibilidades cresce absurdamente conforme aumentamos n .

Figura 56: Tabela.

n (cidades)	Rotas possíveis $(n - 1)! \div 2$
4	$3! \div 2 = 3$
5	$4! \div 2 = 12$
6	$5! \div 2 = 60$
10	$9! \div 2 = 181,440$

12 Capítulo 12: O Que As Máquinas Nunca Farão

Entendeu-se aqui que a questão é saber quais problemas um computador não pode resolver. Se todos os problemas são computáveis ou se alguns são complicados demais pra que um computador resolva.

12.1 Máquina DIN-DIN

É uma máquina que se baseia na máquina de *turing* abordada nos capítulos anteriores. Ela é formada por uma fita de papel dividida em pequenos quadrados e por quatro carimbos: **D**, **I**, **N** e **-**.

Seu objetivo é demonstrar que as máquinas não são capazes de fazer tudo.

Com base nos símbolos citados, podemos fazer várias combinações, mas apenas as sentenças são válidas.

12.1.1 Sentenças

I - X

ID - X

NI - X

NID -X

Sendo que no lugar de **X**, podemos escolher qualquer sequência pra substituir.

O **X** não faz parte dos símbolos formalmente.

I: imprimível

D: dobro

N: não

ND: dobro não

Figura 57: Máquina DIN-DIN.

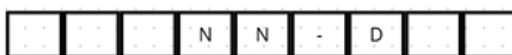


A máquina verifica a fita e procura por algumas sequências.

Exemplo: I-X

É quando a sequência X aparece na fita. Então NN-D é verdadeira.

Figura 58: Máquina DIN-DIN- I-X.



Exemplo: ID-X

É quando a sequência XX aparece duas vezes na fita. Então N é verdadeira.

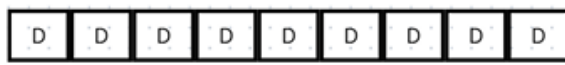
Figura 59: Máquina DIN-DIN- ID-X.



Exemplo: NI-X

É quando a sequência X nunca aparece na fita.

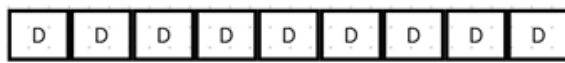
Figura 60: Máquina DIN-DIN- NI-X.



Exemplo: NID-X

É quando a sequência XX nunca aparece duas vezes na fita. Então a Sequência NID-D é falsa pois DD aparece na fita dobrado.

Figura 61: Máquina DIN-DIN- NID-X.



O problema é que não é possível construir uma máquina que só forneça sentenças verdadeiras, pois isso gera um paradoxo e a máquina para. Se tivermos uma fita com a sentença NID-NID por exemplo, é como se estivesse escrito o seguinte:

"A sentença NID-NID nunca será impressa pela máquina."

1- Se NID-NID for verdadeira, significa que ela não será impressa, mas como ela é verdadeira, a máquina deveria imprimir ela, porque ela só imprime verdadeiras.

Há uma contradição, pois ela diz que não será impressa, mas está sendo.

2- Se NID-NID for falsa, significa que é mentira que "ela não será impressa", ou seja, ela será impressa, mas a máquina não imprime sentenças falsas.

A questão é que não tem como essa sentença ser nem verdadeira nem falsa sem gerar um paradoxo.

12.2 Castor Ocupado

Para entender melhor essa parte foi necessário relembra os funcionamentos de uma Máquina de *Turing* que basicamente é uma grade fita infinita dividida em várias células com um 0 ou um 1 em cada uma.

Há também um cabeça que lê os símbolos e dependendo de qual seja, ele pode apagá-lo, escrever um novo símbolo, mover para a esquerda ou direita.

Cada máquina de *Turing* tem um conjunto de regras que determina o que fazer.

De acordo com o que foi estudado, há uma regra especial que diz à máquina quando parar de funcionar.

Uma única máquina pode ter qualquer número fixo de regras a seguir.

Quanto mais regras, mais cálculos complexos ela pode fazer.

A questão é que a partir da regra 1, a máquina pode parar eventualmente ou funcionar para sempre.

Uma máquina de *Turing* pode então, podem ser com parada e sem parada.

Um outro cientista, *Tibor Radó* classificou as máquinas de *Turing* em grupos de acordo com o número de regras.

Em cada grupo, algumas máquinas funcionavam pra sempre, algumas pararam muito rápido e outras demoraram a parar. Ele chamou de *Castor Ocupado*.

Uma delas será a última a parar, então essa seria a mais eficiente.

A pergunta nessa parte da pesquisa então é saber em quanto tempo um programa de computador pode levar pra ser executado antes de parar.

Optou-se aqui, por unir a linguagem do livro disponibilizado com outras fontes de informações. Acredita-se que foi mais fácil para mim adotar o número dos passos dados pelo castor antes de parar de *Busy Beaver* ou *BBn*.

Exemplo: Para uma máquina de Turing de 1 regra

Pra ter certeza que a máquina irá parar, podemos ter apenas a regra de que não importa o que ela leia, ela irá parar.

Portanto, no caso de uma máquina de Turing de regra 1, o *BBn* é igual a 1, pois ela dá apenas 1 passo e para em seguida.

$$BB(1) = 1$$

Existe uma máquina com 11 estado que pega o que estiver na fita e duplica. Por exemplo: se tiver 4 marcas 1s, ela transforma em 8.

Para entender a máquina de 47 estados, foi utilizado um desenho numa folha de papel. Nessa máquina você utiliza o conceito da máquina de 11 estados.

1- Começamos com 25 1s escritos na fita.

2- Dobramos esse número pra 50 com a máquina de estado 11

3- E fazemos isso mais uma vez transformando o 50 em 100. Então a máquina de 47 estado pode escrever pelo menos 100 1s.

Figura 62: $BB(47) = 100$.



Não dá pra um programa calcular qual a maior quantidade de 1's que uma máquina de Turing com n .

estados pode imprimir antes de parar.

Algumas rodam pra sempre, não dá pra prever todas as máquinas e não existe uma que consiga fazer isso.

12.3 O Problema da Parada

Turing fez a seguinte pergunta: "Podemos escrever um programa que nos diga se algum outro irá parar ou ficar rodando pra sempre?"

Um programa é uma sequência de instruções escritas em uma linguagem que o computador pode entender.

Exemplo: Dados 2 valores x e y , x divide y igualmente?

Basta escrever um programa que pega y e divide por x . Se o resto for 0, o programa retorna "*sim*". Se o resultado for diferente de 0, o programa retorna "não". Mas isso só faz sentido se dermos valores a x e y , as entradas ou **inputs**.

Pesquisando sobre esse assunto, foi descoberto que existe uma conjectura chamada Goldbach, que faz a seguinte pergunta: **TODO NÚMERO PAR MAIOR QUE 2 PODE SER ESCRITO COMO A SOMA DE DOIS PRIMOS?**

Embora todos os pares até agora obedeçam essa sentença, não há como provar isso. Por mais que seja uma verdade, ainda é **considerado uma conjectura**.

Pode-se encontrar um programa que possa prever se qualquer outro e sua entrada serão interrompidos ou executados pra sempre?

P é o nome desse programa. **P** diz que se qualquer outro programa em sua entrada irá parar ou entrar em looping, mas se construíssemos outro programa, **MP**, que faz o oposto de **P**?

Se **P** disser que vai parar, **MP** entra em loop infinito, mas se **P** disser que o programa vai rodar pra sempre, **MP** para de rodar.

Se **MP** recebe um programa **MK** como entrada, **MP** faz o oposto de **P**, então ele primeiro precisa saber o que **P** diria sobre **MK**.

Se **P** diz que **MK** para, **MP** roda pra sempre, mas se **P** diz que **MK** roda pra sempre, **MP** para. Mas e se **MP** receber a si mesmo?

MP pergunta a **P** o que **MP(MP)** faria.

Se **P** diz que **MP(MP)** para, **MP** roda pra sempre, mas se **MP** rodar pra sempre, então **P** estava errado, porque **MP(MP)** não para.

Se **MP** para, **MP** roda pra sempre, e se **MP** roda pra sempre, **MP** para.

Entramos numa contradição porque **MP** não pode parar e rodar pra sempre, então **P** não pode existir.

12.4 O Jogo do MU

O objetivo é ir seguindo regras fixas pra transformar a palavra MI em MU.

12.4.1 Regras

1- Se a palavra termina com I, você pode adicionar um U no final.

Exemplo: $MI \rightarrow MIU$

2- Se a palavra começa com M e depois vem uma sequência (qualquer), você pode duplicar essa parte.

Exemplo: $MIU \rightarrow MIUIU$

3- Se houver três I's seguidos (III), pode trocar isso por um U.

Exemplo: $MIII \rightarrow MU$ Se tiver dois U's seguidos (UU), você pode apagá-los.

4- Se tiver dois U's seguidos (UU), você pode apagá-los.

Exemplo: $MUUU \rightarrow MU$

O problema é que não dá pra chegar em MU.

Tudo depende da quantidade de I's:

- Começamos com 1 apenas.

- Pra que os I's saiam, a sua quantidade dever ser múltiplo de 3, mas nenhuma regra consegue fazer isso.

12.5 Copinhos de Café

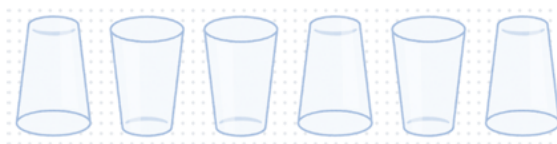
12.5.1 Objetivo

Deixar todos os copinhos com a boca para cima (todos $+1$), virando sempre 2 copinhos de cada vez.

($+1$): boca pra cima

(-1): boca pra baixo

Figura 63: Desafio do Copo.



Há 3 copos virados para baixo.

Toda vez que viramos 2 copos, o número de copos virados para baixo muda em número par.

Como há 3 copos virados pra baixo (n° ímpar), nunca chegaremos a 0 (n° par).

Para essa configuração, não há solução.

13 Considerações Finais

Conclui-se que os estudos desenvolvidos permitiram ampliar a compreensão sobre lógica matemática, computação e inteligência artificial, relacionando conceitos teóricos com exemplos práticos. A pesquisa contribuiu para o desenvolvimento do raciocínio lógico e para uma melhor compreensão do funcionamento das máquinas e dos sistemas computacionais.

14 Agradecimento

"Agradeço ao Conselho Nacional de Desenvolvimento Científico e Tecnológico (CNPq) pelo apoio financeiro e pelo incentivo à pesquisa, fundamentais para a realização deste trabalho sobre Inteligência Artificial e Lógica Matemática."

15 Referências Das Pesquisas

15.1 Jogo NIM

<https://youtu.be/-99F4as2V6c?si=XzpJboGeMYNJSBJw>

<https://youtu.be/3COOMrAmpMA?si=S12RQsgJRvsrBJfO>

<https://youtu.be/94eby0vJPYQ?si=ibpZPv-wemHU0nGW>

15.2 Meio Somador

<https://youtu.be/P8GzVkOcEH4?si=WlcDsu5pOK85U1Y>

<https://youtu.be/h-mZVC-oWcI?si=KQbjA4xYfuiSwkPx>

15.3 Linguagens

<https://youtu.be/4zMwOozUt9U?si=Q4cCyJ4nBE-jj-lr>

15.4 Autômatos Finitos Determinísticos

<https://youtu.be/0W-HK4MYKL8?si=oRbNYuwSaDvSTrUm>

<https://youtu.be/OAOt9BQpSBE?si=a7pbLOsoFGkzoBPf>

<https://youtu.be/mCwQoM8KaZk?si=oREy0dDbbST-acRA>

15.5 Relês

<https://youtu.be/StBCiTJfG4k?si=Bq1SquRORY-sPoVE>

<https://youtu.be/ezBKY-ujJLQ?si=SISdHbwSXufboxCM>

<https://youtu.be/u4pmzc6bUt0?si=FvMSqcJxd2S-mJmn>

15.6 As Máquinas de Turing

<https://youtu.be/t37GQgUPa6k?si=8g80F41Q00DhLEbe>

<https://youtu.be/SgzrgOMf5Ts?si=6mVntUpCYanFLIEw>

<https://youtu.be/macM-MtS-w4?si=5d1Dr645r1mwmJ5j>

<https://youtu.be/Ccla4Mnoo9s?si=-llp5beSG9-49CU7>

<https://youtu.be/E0YX8BC4RLo?si=HCl6P039QTWC03M3>

<https://youtu.be/CE8UhcyJS0I?si=g-pqgzA4TZUesBk5>

<https://youtu.be/dNRDvLACg5Q?si=6omUFmjBuTuM2vL1>

<https://youtu.be/O-OLFG0WJpc?si=-nIH6t2U07iCoeRX>

15.7 Autômatos Celulares

<https://youtu.be/D5D0uvKiJGU?si=ny1ksYtaYxsf9hvp>

<https://youtu.be/mDQyDfSGakQ?si=o67UFIWmhmGAxrAk>